

Patrones de diseño

AutoRent Express S.A. — plataforma de renta de vehículos

Framework: Django 6.0.7 (Python) · PostgreSQL

Patrones implementados: State · Strategy · Factory Method

Dominio: ciclo de vida de la reserva y cálculo de tarifas

Repositorio: AYD2_A_2S2026_PRACTICA1_G2

Tres patrones, un solo módulo

Los tres patrones se concentran en la app `reservas`, que es donde vive la lógica de negocio del enunciado: la propuesta del agente, la respuesta del cliente y el precio de la renta.

State

DE COMPORTAMIENTO

El ciclo de vida de la reserva: qué transición es legal en cada momento.

`reservas/estados.py`

Strategy

DE COMPORTAMIENTO

Tres formas intercambiables de calcular el total de una renta.

`reservas/calculos.py`

Factory Method

CREACIONAL

Traduce el plan guardado en la base al algoritmo de cálculo concreto.

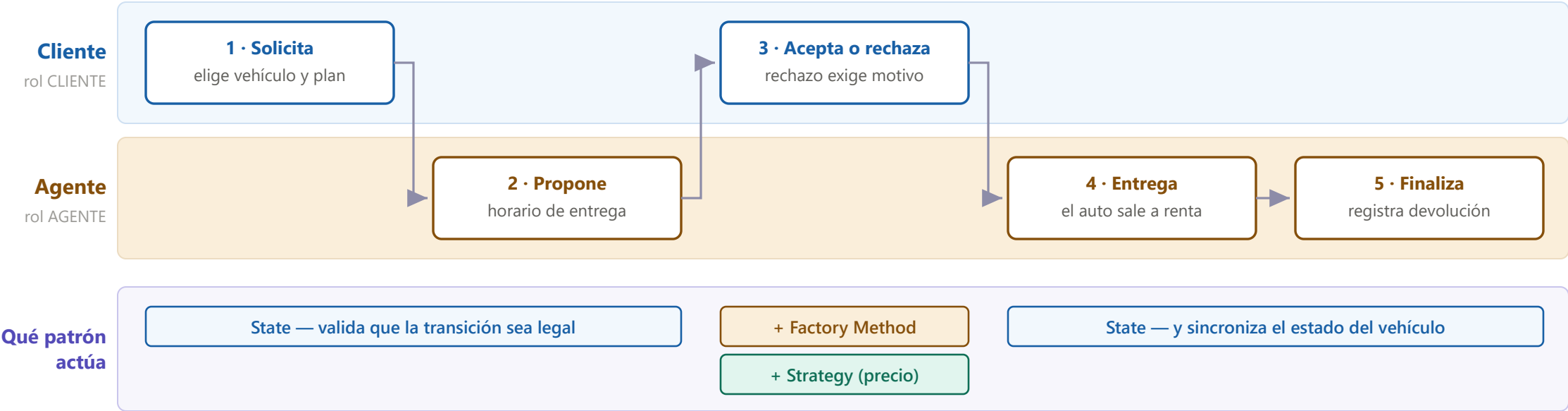
`reservas/models.py`

LA IDEA QUE AMARRA TODO

No son tres patrones sueltos puestos para cumplir el requisito. Se activan en cadena en una sola operación: cuando el cliente **acepta** la propuesta, el **State** decide que la acción es legal, el **Factory** entrega el algoritmo y la **Strategy** calcula el precio.

El recorrido de una renta, y quién actúa en cada paso

El enunciado define tres roles. La lógica interesante ocurre entre **cliente** y **agente**: ninguno de los dos puede avanzar solo, se van pasando el turno. Eso es lo que el patrón State vigila.

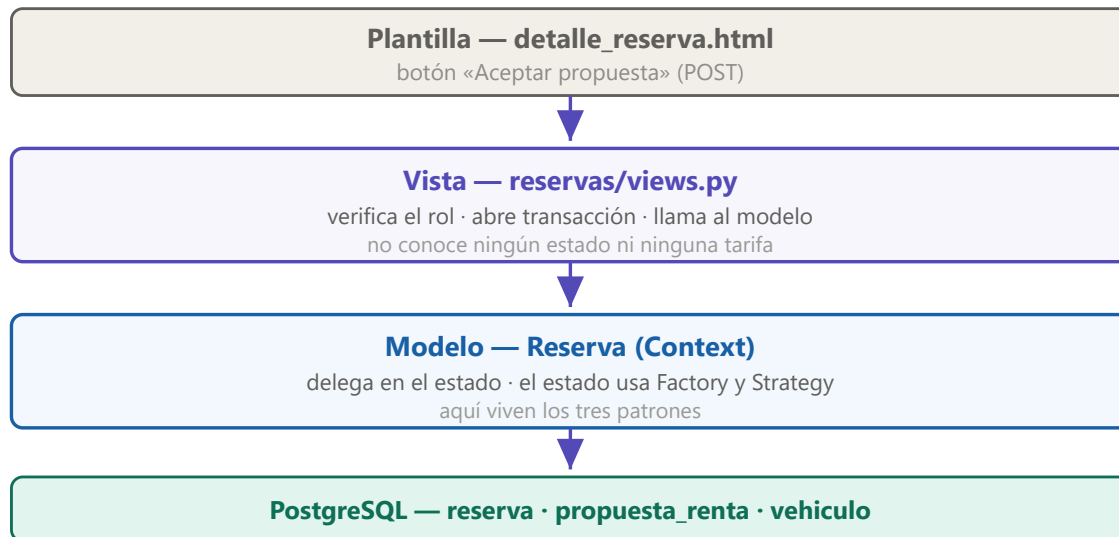


Los tres patrones se concentran en el paso 3, cuando el cliente acepta: es el único momento en que se calcula dinero.

Del clic del usuario a la clase de estado

Django organiza el código en **MTV**: plantilla, vista y modelo. Los patrones viven en el **modelo**, no en la vista. La vista sólo traduce el clic en una llamada de negocio y muestra el error si lo hay.

LA CADENA COMPLETA



LA VISTA, EN SU TOTALIDAD

```

# reservas/views.py
@login_required
@require_POST
@transaction.atomic
def aceptar_reserva(request, reserva_id):
    if not es_cliente(request.user):
        return HttpResponseForbidden('Solo el cliente puede aceptar.')

    reserva = get_object_or_404(
        Reserva.objects.select_for_update(), # bloqueo de fila
        pk=reserva_id, cliente_usuario=request.user)

    try:
        reserva.aceptar() # ← toda la regla vive aquí
        messages.success(request, 'Reserva aceptada.')
    except ValidationError as error:
        messages.error(request, ' '.join(error.messages))

    return redirect('reservas:detalle', reserva_id=reserva.id_reserva)
  
```

POR QUÉ ESTO IMPORTA PARA LA DEFENSA

La vista tiene **una sola línea de negocio**. Si mañana cambia la regla de qué estado puede aceptar, la vista no se toca. Ese es el beneficio

concreto de haber puesto los patrones en el modelo y no en la vista.

DETALLE DE ROBUSTEZ

Corre dentro de `transaction.atomic` y toma la fila con `select_for_update()` . Si dos peticiones intentaran avanzar la misma reserva al mismo tiempo, la segunda espera: **no hay dos transiciones simultáneas.**

State

PATRÓN DE COMPORTAMIENTO · GOF

Permite que un objeto altere su comportamiento cuando cambia su estado interno.

El objeto parecerá cambiar de clase.

`reservas/estados.py` · contexto en `reservas/models.py`

Cinco operaciones, la misma cadena de if

SIN EL PATRÓN

```
class Reserva:
    def aceptar(self):
        if self.estado == 'PROPUESTA':
            ...
        else:
            raise Error('no se puede')

    def rechazar(self, motivo):
        if self.estado == 'PROPUESTA':
            ...
        else:
            raise Error('no se puede')

    def entregar(self):
        if self.estado == 'ACEPTADA': ...
    def finalizar(self):
        if self.estado == 'EN_RENTA': ...
    def proponer(self, horario):
        if self.estado == 'SOLICITADA': ...
```

POR QUÉ DUELE

- La regla de negocio queda **repartida en cinco métodos**. Para saber qué se puede hacer en el estado `PROPUESTA` hay que leerlos todos.
- Agregar un estado nuevo obliga a **tocar los cinco**. Es exactamente lo contrario al principio abierto/cerrado.
- Nada garantiza que las cinco cadenas de condicionales sean **coherentes entre sí**. Basta olvidar un `else` para permitir una transición ilegal.

APLICABILIDAD — GOF

«Úsese State cuando las operaciones tienen **condicionales grandes y de varias partes** que dependen del estado del objeto [...] y frecuentemente **varias operaciones contienen la misma estructura condicional**.»

Gamma, Helm, Johnson & Vlissides — Design Patterns, patrón State.

Es literalmente el caso: cinco operaciones con la misma estructura condicional sobre el campo `estado`.

Un estado, una clase

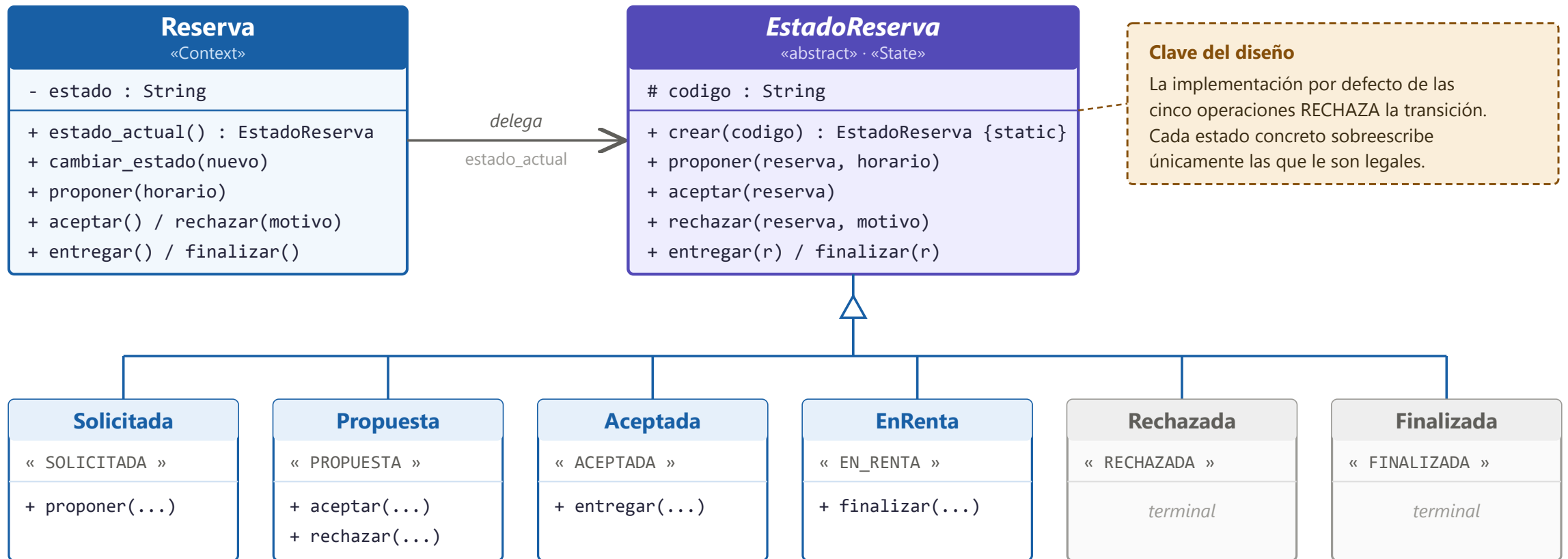


Diagrama de clases UML del patrón State. **Reserva** es el Context; **EstadoReserva** es el State; las seis clases de abajo son los ConcreteState.

La máquina de estados de una reserva

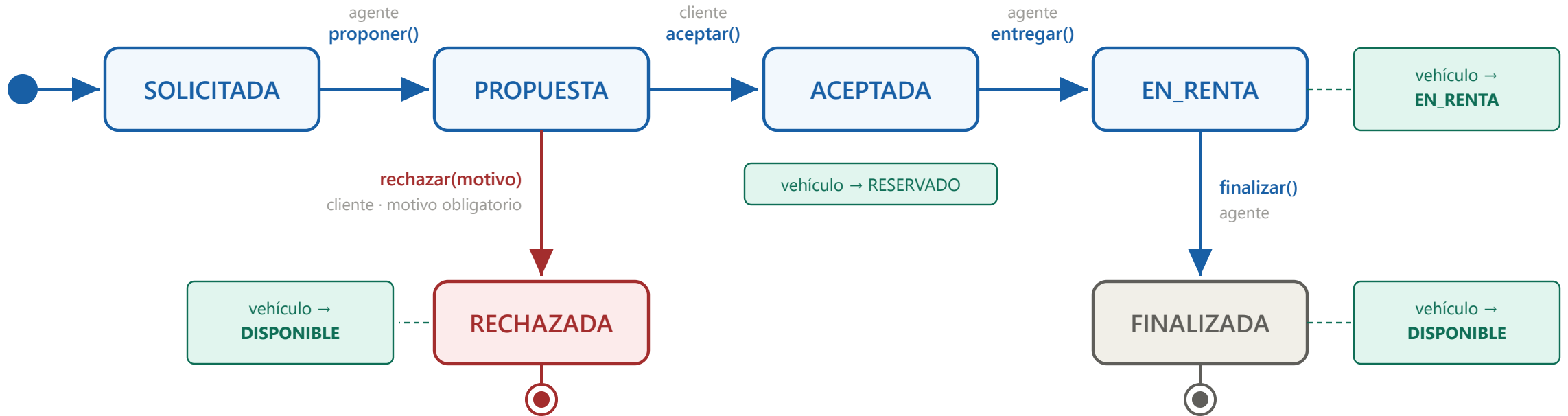


Diagrama de máquina de estados UML. En verde, el efecto colateral sobre el vehículo que cada transición dispara. **RECHAZADA** y **FINALIZADA** son terminales.

La clase abstracta que prohíbe todo

```
# reservas/estados.py
class EstadoReserva(ABC):
    codigo = None

    @staticmethod
    def crear(codigo):
        # del código persistido al ConcreteState
        return _ESTADOS[codigo]()

    def proponer(self, reserva, horario): self._invalido('proponer')
    def aceptar(self, reserva):           self._invalido('aceptar')
    def rechazar(self, reserva, motivo): self._invalido('rechazar')
    def entregar(self, reserva):          self._invalido('entregar')
    def finalizar(self, reserva):         self._invalido('finalizar')

    def _invalido(self, accion):
        raise ValidationError(
            f'No se puede {accion} una reserva '
            f'en estado {self.codigo}.')
```

Código tal cual está en el repositorio (simplificado sólo en el salto de línea de las cinco firmas).

LA DECISIÓN DE DISEÑO

La clase base **no** declara métodos abstractos vacíos. Declara métodos que **lanzan excepción**. El comportamiento por omisión es «esta transición es ilegal».

Consecuencia directa: un estado concreto sólo escribe los métodos que **sí** puede ejecutar. Todo lo demás queda prohibido sin escribir una sola línea.

```
class Rechazada(EstadoReserva):
    """Estado terminal: no acepta ninguna transición."""
    codigo = 'RECHAZADA'
```

Tres líneas y el estado queda completamente blindado: las cinco operaciones heredadas rechazan.

QUÉ SE GANÓ

Las reglas del negocio dejaron de estar repartidas en condicionales. Ahora **se leen en la definición de cada clase**: lo que una clase implementa es lo que ese estado permite.

El estado concreto y el contexto

CONCRETESTATE — RESERVAS/ESTADOS.PY

```
class Propuesta(EstadoReserva):
    codigo = 'PROPUESTA'

    def aceptar(self, reserva):
        reserva.precio_dia_aplicado = reserva.vehiculo.precio_dia
        reserva.descuento_aplicado = reserva.plan.descuento
        reserva.precio_total = reserva.calcular_total()
        reserva.vehiculo.estado = Vehiculo.Estado.RESERVADO
        reserva.vehiculo.save()
        reserva.cambiar_estado(Aceptada())

    def rechazar(self, reserva, motivo):
        if not motivo or not motivo.strip():
            raise ValidationError('Debe indicar el motivo.')
        reserva.motivo_rechazo = motivo
        reserva.vehiculo.estado = Vehiculo.Estado.DISPONIBLE
        reserva.vehiculo.save()
        reserva.cambiar_estado(Rechazada())
```

CONTEXT — RESERVAS/MODELS.PY

```
class Reserva(models.Model):
    estado = models.CharField(choices=Estado.choices)

    @property
    def estado_actual(self) -> EstadoReserva:
        return EstadoReserva.crear(self.estado)

    def cambiar_estado(self, nuevo: EstadoReserva):
        self.estado = nuevo.codigo
        self.save()

    # Las cinco operaciones sólo DELEGAN
    def proponer(self, horario): self.estado_actual.proponer(self, horario)
    def aceptar(self):           self.estado_actual.aceptar(self)
    def rechazar(self, motivo): self.estado_actual.rechazar(self, motivo)
    def entregar(self):         self.estado_actual.entregar(self)
    def finalizar(self):        self.estado_actual.finalizar(self)
```

EL PUNTO A DESTACAR

En el Context **no quedó ni un solo if** sobre el estado. Cada método es una línea que delega. El objeto de estado se materializa desde el código persistido en la columna `estado` y se descarta después.

La misma regla, en dos capas

Las reglas del patrón no viven sólo en Python: están espejadas en los `CHECK` del esquema. Si alguien insertara por SQL directo, la base también las hace cumplir.

Constraint en el DDL	Regla equivalente en el patrón	Dónde se aplica
chk_estado_reserva	Sólo existen los seis códigos que conoce el diccionario <code>_ESTADOS</code> .	<code>EstadoReserva.crear()</code>
chk_motivo_si_rechazada	No se puede rechazar sin motivo: se valida antes de cambiar de estado.	<code>Propuesta.rechazar()</code>
chk_horario_si_propuesta	Se asigna <code>horario_propuesto</code> antes de pasar a <code>PROPUESTA</code> .	<code>Solicitada.proponer()</code>

EFFECTO COLATERAL CONTROLADO

Cada transición además sincroniza el estado del vehículo (`RESERVADO` , `EN_RENTA` , `DISPONIBLE`). Al estar dentro de la clase de estado, es imposible cambiar la reserva y olvidar el vehículo.

POR QUÉ SE PERSISTE UN CÓDIGO Y NO EL OBJETO

El objeto de estado **no tiene datos propios**, sólo comportamiento. Por eso se guarda únicamente el discriminador `estado` y el `ConcreteState` se reconstruye cuando se necesita. Es el consejo del GoF: los objetos de estado sin atributos pueden compartirse.

Strategy

PATRÓN DE COMPORTAMIENTO · GOF

Define una familia de algoritmos, encapsula cada uno y los hace intercambiables.
Permite que el algoritmo varíe independientemente de los clientes que lo usan.

```
reservas/calculos.py
```

Tres formas de cobrar la misma renta

El enunciado pide **planes de renta**. Cada plan cobra distinto: por día, por semana con descuento, por mes con descuento.

SI NO SE USARA EL PATRÓN

`Reserva.calcular_total()` tendría un `if` por plan, y agregar un «Plan quincenal» obligaría a modificar la clase `Reserva` — una clase que no tiene nada que ver con fórmulas de precio.

CON EL PATRÓN

Cada fórmula es una clase con un solo método. `Reserva` recibe una estrategia y la ejecuta **sin saber cuál es**. Agregar un plan nuevo es agregar una clase.

APLICABILIDAD — GOF

«Úsese *Strategy* cuando muchas clases relacionadas difieren sólo en su comportamiento» y «cuando se necesitan distintas variantes de un algoritmo».

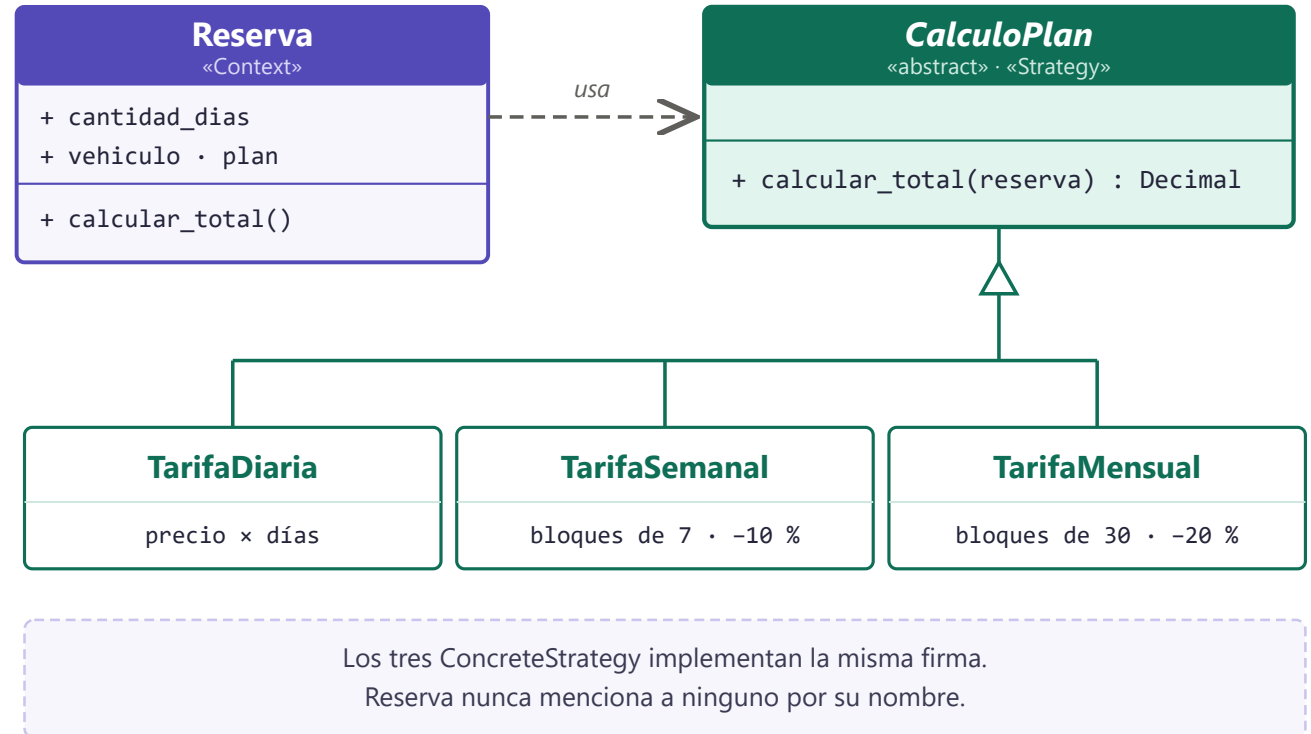


Diagrama de clases UML del patrón Strategy.

Una fórmula, una clase

```
# reservas/calculos.py
class CalculoPlan(ABC):
    """Strategy: interfaz común a todas
    Las formas de calcular el total."""

    @abstractmethod
    def calcular_total(self, reserva) -> Decimal: ...

class TarifaDiaria(CalculoPlan):
    def calcular_total(self, reserva) -> Decimal:
        return reserva.vehiculo.precio_dia * reserva.cantidad_dias

class TarifaSemanal(CalculoPlan):
    def calcular_total(self, reserva) -> Decimal:
        precio = reserva.vehiculo.precio_dia
        semanas, sueltos = divmod(reserva.cantidad_dias, 7)
        factor = Decimal('1') - (reserva.plan.descuento / 100)
        return (semanas * 7 * precio * factor) + (sueltos * precio)
```

`TarifaMensual` es idéntica cambiando 7 por 30.

CÓMO LA USA EL CONTEXT

```
# reservas/models.py - clase Reserva
def calcular_total(self) -> Decimal:
    estrategia = self.plan.calcular_total() if self.plan else TarifaDiaria()
    return estrategia.calcular_total(self)
```

PLANES SEMBRADOS EN LA BASE

Plan	calculo_codigo	Días mín.	Descuento
Plan Diario	DIARIA	1	0 %
Plan Semanal	SEMANAL	7	10 %
Plan Mensual	MENSUAL	30	20 %

DETALLE DE NEGOCIO

El descuento aplica **sólo sobre bloques completos**. En el plan semanal, 10 días se cobran como 7 días con descuento más 3 días a tarifa plena. Esa regla vive en una sola clase.

Factory Method

PATRÓN CREACIONAL · GOF

Define una interfaz para crear un objeto, pero deja que sea la implementación la que decida qué clase concreta instanciar.

```
PlanRenta.calcular() · reservas/models.py
```

¿Quién decide cuál Strategy se usa?

Strategy resolvió *cómo* se calcula. Queda un hueco: el plan que eligió el cliente está guardado en la base como un **texto** ('SEMANAL'). Alguien tiene que traducir ese texto a un objeto.

EL RIESGO

Si esa traducción la hiciera `Reserva`, el Context volvería a conocer los nombres de las clases concretas — justo lo que Strategy vino a evitar.

LA SOLUCIÓN

`PlanRenta` actúa como **Creator**. Su método `calcula()` es el Factory Method: recibe el discriminador que él mismo tiene persistido y devuelve el producto concreto.

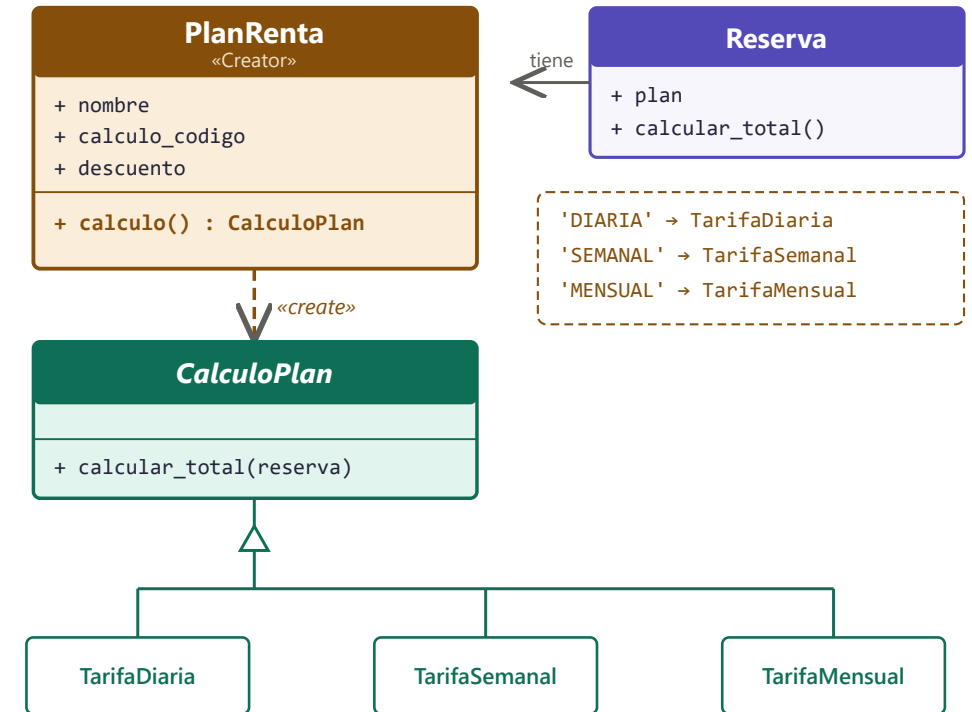
EL CÓDIGO

```
# reservas/models.py
class PlanRenta(models.Model):

    _PRODUCTOS = {
        Calculo.DIARIA: TarifaDiaria,
        Calculo.SEMANAL: TarifaSemanal,
        Calculo.MENSUAL: TarifaMensual,
    }

    calculo_codigo = models.CharField(choices=Calculo.choices)
```

Patrones de diseño - AutoRent Express S.A.



El Creator conoce a los productos; `Reserva` no.


```
# ----- FACTORY METHOD -----  
def calculo(self) -> CalculoPlan:  
    return self._PRODUCTOS[self.calculo_codigo]()
```

LOS TRES JUNTOS

Qué pasa cuando el cliente presiona «Aceptar»

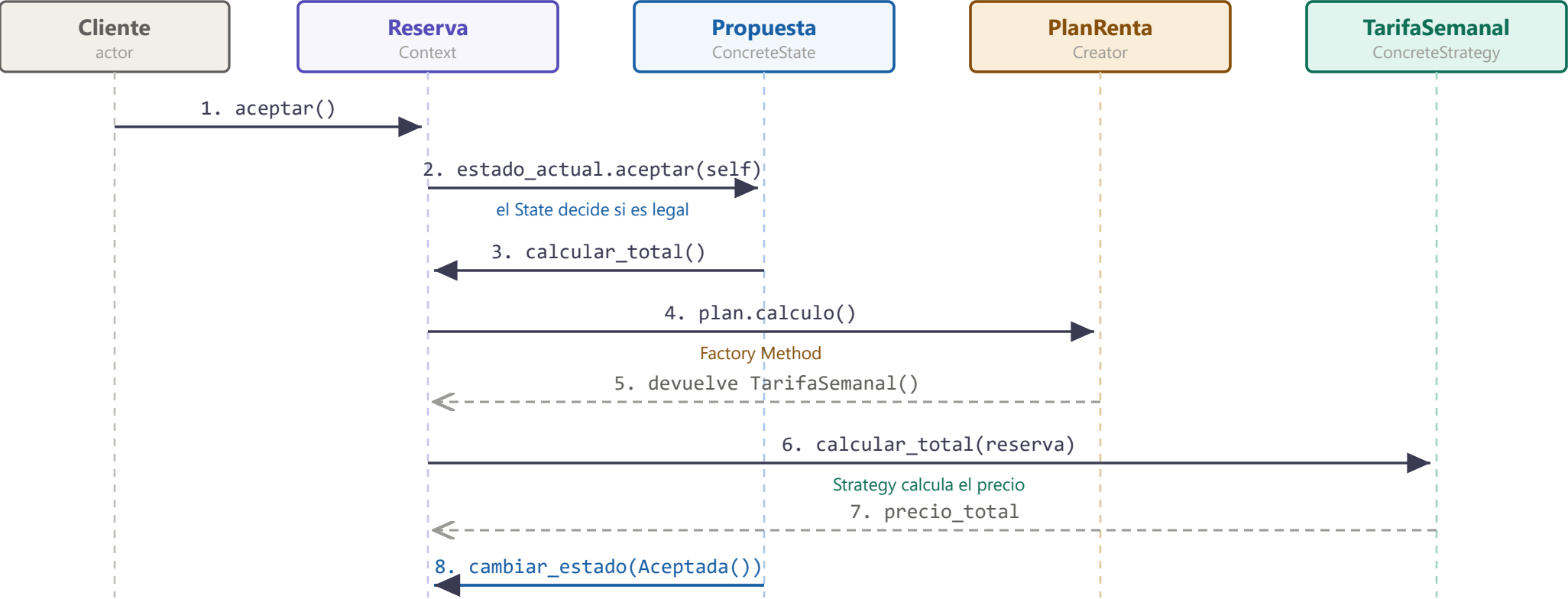


Diagrama de secuencia UML de `Reserva.aceptar()`. Además, en el paso 8 el vehículo pasa a `RESERVADO`.

EL REPARTO DE RESPONSABILIDADES

State decide si la acción es legal y qué sigue · **Factory Method** decide quién calcula · **Strategy** decide cómo se calcula. Ninguno invade al otro.

Si el diagrama es igual, ¿por qué son dos patrones?

State y Strategy tienen la **misma estructura**: un contexto que delega en una jerarquía de clases. Lo que los separa es la **intención**, y eso se ve en el proyecto.

	State — reservas/estados.py	Strategy — reservas/calculos.py
Qué encapsula	Una situación del objeto y las transiciones legales desde ella.	Un algoritmo completo e independiente.
Quién elige	Lo elige el propio patrón : <code>Propuesta.aceptar()</code> decide que el siguiente estado es <code>Aceptada</code> .	Lo elige el cliente / la configuración : el plan que el usuario seleccionó.
Se conocen entre sí	Sí . <code>Propuesta</code> nombra a <code>Aceptada</code> y a <code>Rechazada</code> .	No . <code>TarifaSemanal</code> no sabe que existe <code>TarifaMensual</code> .
Cambia solo	Sí , durante la vida del objeto. Una reserva recorre varios estados.	No . Se fija al crear la reserva y no cambia.
Frase resumen	«El objeto está en un estado.»	«El objeto usa un algoritmo.»

CÓMO SE RESPONDE EN UNA LÍNEA

Una reserva **pasa** de SOLICITADA a PROPUESTA sola, por efecto de una operación. Nunca «pasa» de TarifaDiaria a TarifaSemanal: esa se elige una vez y se queda.

Preguntas que conviene tener respondidas

¿ESE FACTORY METHOD ES EL DEL GOF?

Es la variante que el propio GoF llama **factory method parametrizado**: un solo método recibe un identificador y devuelve el producto, en lugar de una subclase por producto. Se eligió así porque el discriminador ya existía como columna en la base (`plan_renta.calculo_codigo`) y crear una subclase de `PlanRenta` por cada plan habría duplicado la tabla sin ganar nada.

¿Y `ESTADORESERVA.CREAR()` ?

Es el mismo mecanismo aplicado a los estados: traduce el código persistido al ConcreteState. Es la pieza que conecta la **base de datos relacional** con el patrón, porque una tabla no puede guardar un objeto de comportamiento.

¿QUÉ PASA SI INTENTO UNA TRANSICIÓN ILEGAL?

Se lanza `ValidationError` con el mensaje «No se puede aceptar una reserva en estado `EN_RENTA`». La vista lo captura y lo muestra como mensaje de error. Nunca se corrompe el dato: las operaciones críticas corren dentro de `transaction.atomic` .

¿CÓMO AGREGARÍAN UN PLAN QUINCENAL?

Tres pasos, sin tocar `Reserva` ni las vistas: crear la clase `TarifaQuincenal(CalculoPlan)` , agregar la entrada al diccionario `_PRODUCTOS` y sembrar la fila en `plan_renta` . Ese es el argumento concreto de por qué los patrones valieron la pena.

¿UN ESTADO NUEVO?

Una clase que hereda de `EstadoReserva` , sobrescribe sólo lo legal y se registra en `_ESTADOS` . Las clases existentes no se modifican.

CIERRE

Resumen

Patrón	Categoría	Problema real que resolvió	Archivo
State	Comportamiento	Cinco operaciones con la misma cadena de condicionales sobre el campo estado .	reservas/estados.py
Strategy	Comportamiento	Tres fórmulas de tarifa que deben poder intercambiarse y crecer.	reservas/calculos.py
Factory Method	Creacional	Traducir el plan persistido en la base al objeto de cálculo correcto.	reservas/models.py

SIN CONDICIONALES

El Context quedó sin un solo if sobre el estado: cinco métodos que sólo delegan.

ABIERTO A EXTENSIÓN

Un plan nuevo o un estado nuevo se agregan escribiendo una clase, sin modificar las existentes.

COHERENTE CON LA BASE

Las reglas del patrón están espejadas en los CHECK del DDL: no dependen de una sola capa.

REFERENCIA

Gamma, E., Helm, R., Johnson, R. & Vlissides, J. — *Design Patterns: Elements of Reusable Object-Oriented Software*. Patrones **State**, **Strategy** y **Factory Method**.